Conceptual Exploration of Sudoku

Rules of solved and partial grids: implementation, results, and directions

The `conceptual-exploration` library

October 2026

Abstract

This document describes how the `conceptual-exploration` library explores Sudoku. It is written for a reader who knows attribute and rule exploration as presented in Ganter and Obiedkov's *Conceptual Exploration*, and it concentrates on what is specific to this application. Sudoku is explored in three ways: with first-order rules about *solved* grids, where the exploration rediscovers the constraints of the game; with first-order rules about *partial* grids, where the predicates say what follows from the givens and the rules found are solving techniques such as naked and hidden singles and locked candidates; and propositionally, with cell assignments as attributes. Each needs its own expert. Solved grids are decided by a SAT encoding (or by the Z3 SMT solver), partial 4×4 grids by listing all 288 solved grids, and partial grids of any size by counterexample-guided refinement with two SAT solvers. The document also describes the features of the general library that the application led to: declared properties of predicates, variable substitutions whose images may be truth values, bit-set closures, and the simplified form in which implications are printed. It closes with a user's guide, the limits met so far, and directions for further work.

Contents

1 Introduction **2**

2 Rule exploration as implemented **3**
- 2.1 The exploration loop . . . 3
- 2.2 Atoms, variables and objects . . . 3
- 2.3 Variable maps . . . 3
- 2.4 Declared properties of predicates . . . 4
- 2.5 Data structures . . . 4
- 2.6 Reading the result . . . 5

3 Solved grids **5**
- 3.1 Vocabulary . . . 5
- 3.2 Background knowledge . . . 5
- 3.3 Experts . . . 6
- 3.4 Results . . . 7

4 Partial grids **7**
- 4.1 Objects and completion semantics . . . 7
- 4.2 Confinement predicates . . . 8
- 4.3 Background knowledge . . . 8
- 4.4 The enumerating expert . . . 9
- 4.5 The CEGAR expert . . . 9
- 4.6 Results . . . 10

5 Propositional exploration **11**

6 How the results were checked **12**

7 User's guide **12**
7.1 Running an exploration 12
7.2 Configuration files 13
7.3 From Python 13
7.4 Where the code is 14

8 Limits met so far **14**

9 Directions for further development **14**

A Rules of the four-cell partial-grid exploration **15**

B Rules of locked candidates on 9×9 grids **20**

1 Introduction

A Sudoku grid of *block size* k has $n = k^2$ rows and n columns. Its n^2 cells are partitioned into n *rows*, n *columns* and n *blocks* of $k \times k$ cells; these are the *units*. Rows are grouped into k *bands* of k consecutive rows, and columns into k *stacks*; a block is the intersection of a band and a stack. A *solved grid* assigns a digit from $\{1, \ldots, n\}$ to every cell so that every unit contains every digit exactly once; a *partial grid* assigns digits, the *givens*, to some cells, and a *completion* of it is a solved grid that agrees with its givens. The classic puzzle has $k = 3$; most experiments below use $k = 2$ as well, where there are 288 solved grids and everything can be checked by brute force.

The aim is to find, by conceptual exploration, the implicational knowledge about Sudoku that can be expressed in a given vocabulary, and to see what kinds of knowledge different vocabularies give access to. Three explorations are implemented.

Solved grids (Section 3). First-order rules over variables for cells and digits, with predicates such as *same row* and *same value*. Given what holds in any grid whatever its digits, the exploration finds exactly the constraints of the game: two cells holding the same digit in a row, column or block are the same cell.

Partial grids (Section 4). First-order rules whose predicates speak about what follows from the givens: a digit is *forced* in a cell if every completion has it there, *excluded* if none does. Rules between such statements are solving techniques; the exploration finds naked and hidden singles, elimination and locked candidates, and the X-wing can be stated and confirmed.

Propositional (Section 5). The attributes are the assignments "cell (r, c) holds digit d" of the 4×4 grid, the constraints of the game are background knowledge, and the symmetry group of the grid acts on everything. Four implications are found, each a concrete deduction.

Section 2 describes the rule exploration of the general library, in particular the features that this application led to. Sections 3–5 describe the three explorations, their experts and their results. Section 6 explains how the results were checked, Section 7 how to run explorations, Section 8 what does not work (yet), and Section 9 how the project could develop.

A companion document, *Conceptual Exploration of Equational Theories of Magmas*, describes the general exploration loop in more detail, including its handling of experts that cannot always decide; that part plays no role here, since all Sudoku experts decide every question.

2 Rule exploration as implemented

2.1 The exploration loop

The library implements attribute exploration as in the book: premises are the sets closed under the implications known so far, enumerated with NextClosure in lectic order, and for each premise P the expert is asked about $P \to P^\diamond \setminus P$, where $P^\diamond$ is the largest set of attributes not refuted by the objects at hand. Objects are partial rows (g^+, g^-); in all Sudoku explorations the experts return complete rows (the option `evaluate_all`), so $P^\diamond$ is the ordinary closure P'' of the context. Background implications and a set Φ of attribute maps ("mappings") complete the picture: accepted implications and added objects are added together with their images under every $\varphi \in \Phi$. The companion document describes this loop in full.

2.2 Atoms, variables and objects

Rule exploration in the sense of the book [1][1] is attribute exploration in which the attributes are *atoms* $p(v_1, \ldots, v_m)$ over a finite set V of variables, and an object is a structure together with an assignment of elements to the variables; it has the atom $p(\vec{v})$ if the structure satisfies p for the assigned elements. An implication $P \to C$ between atoms then stands for the universal Horn sentence

$$\forall \vec{v} \left(\bigwedge P \;\to\; \bigwedge C \right),$$

and it holds in the class of structures exactly if no structure with an assignment refutes it.

In the library, `RuleExploration` takes a list of predicates and of variables and builds the atoms (`atoms_over`): every predicate applied to every tuple of variables of the right length. Variables may be *sorted* (`SortedVariable`); a predicate declares the sorts of its argument positions, and only atoms of matching sorts are built. The Sudoku explorations have two sorts, `CELL` and `NUMBER`. The structure of an object is a grid (solved or partial), and the assignment sends cell variables to cells and number variables to digits.

2.3 Variable maps

A map $\sigma \colon V \to V$ that respects sorts induces a map on atoms, $p(v_1, \ldots, v_m) \mapsto p(\sigma v_1, \ldots, \sigma v_m)$, and a rule that holds remains true when its variables are renamed or identified. `RuleExploration` therefore uses as mappings the maps induced by sort-respecting maps σ: all *permutations* of the variables of each sort, or, with `substitutions=True` (always used for Sudoku), all maps of each sort into itself. With c cell and d number variables there are $c^c d^d - 1$ non-identical maps; for three cells and two numbers, 107.

On objects, a map acts the other way: the image of an object with assignment α under σ is the object with assignment $\alpha \circ \sigma$, which has the atom $p(\vec{v})$ exactly when the original object has $p(\sigma\vec{v})$. This is the copy with row $(\varphi^{-1}(g^+), \varphi^{-1}(g^-))$ of the general loop. Non-injective substitutions are what make rule exploration efficient: a rule accepted for distinct variables is at once specialized to every identification of them, and an object is at once available with all of its assignments that identify variables.

The general loop maps each implication and each object once by each mapping, never by a composition; the set of mappings must therefore be closed under composition. The maps of all substitutions (a monoid) and of all permutations (a group) both are.

Images that are truth values. A substitution may send an atom to an atom whose truth is fixed: identifying x and y sends $\mathsf{SameRow}(x, y)$ to $\mathsf{SameRow}(x, x)$, which every object has.

[1] To check: that the book's formulation (objects as structures with variable assignments, substitutions of variables) matches this description

When predicates declare such facts (next subsection), the library does not keep these atoms as attributes, and a mapping may send an attribute to a truth value (`Truth.TRUE` or `Truth.FALSE`). The image of an implication is then formed as follows: a true atom drops out of either side; a false atom in the premise makes the image say nothing, and it is skipped; a false atom in the conclusion means that the image of the premise cannot hold, and the image concludes every attribute. In the image of an object, an attribute sent to a truth value takes that value.

2.4 Declared properties of predicates

A binary predicate can declare itself *symmetric*, *reflexive* or *irreflexive*, and name its *complement*, a predicate that holds exactly where it does not. These declarations do three things.

1. *Canonical atoms.* Of the two atoms $p(x, y)$ and $p(y, x)$ of a symmetric predicate, only the one with sorted arguments is an attribute; $p(x, x)$ is not an attribute if p is reflexive or irreflexive. `Atom.normalize` maps every atom to its canonical form or to its truth value, and the variable maps normalize their images, so that the image of an attribute is always an attribute or a truth value.

2. *Normalized background.* Background implications may be written without regard to canonical forms; `normalize_implication` rewrites them over the canonical atoms (with the same rules for truth values as for images) and drops those that become trivial.

3. *Complements.* For each pair of complementary predicates, the background is extended by $\{p(\vec{v}), q(\vec{v})\} \to M$ for every canonical atom, where M is the set of all attributes: no object has both. Horn implications cannot say that one of the two holds; that half of the complement relation remains implicit, but the experts respect it.

In the solved-grid exploration with three cell and two number variables, declared properties reduce the attributes from 73 to 28.

2.5 Data structures

Attribute sets, implications and rows are integer bit masks over a numbering of the attributes. Two closures are needed at every step of NextClosure: the closure $\mathcal{L}(X)$ under the implications, and the context closure $X^\diamond$. Both are computed on *transposed* bit sets.

- For the implications, every attribute m carries a bit mask π_m over the implications (by number) whose premise contains m, and a mask κ_m over those whose conclusion contains it. One round of the closure of X computes the implications that can fire, $F = \overline{\bigcup_{m \notin X} \pi_m}$, and adds every $m \notin X$ with $\kappa_m \cap F \neq \emptyset$; rounds are repeated until nothing is added. Ruling out the implications whose premise has an absent attribute is similar to Wild's closure algorithm; the conclusion masks are added so that what fires is read off per attribute as well. A round costs a big-integer operation per attribute rather than a step per implication.

- For the context, the distinct rows are numbered (objects sharing a row share an entry), and every attribute carries masks of the rows that have it and of those that lack it. The rows containing X are the intersection of the masks of the attributes of X, and an attribute belongs to $X^\diamond$ unless one of these rows lacks it.

Accepting an implication adds its images under all mappings before the objects are brought up to date, and complete objects on which no new implication fires are not touched. An implication the theory already contains is recognized in constant time and not mapped again; with mappings closed under composition its images are already present, so a background made of whole orbits is mapped once per orbit.

2.6 Reading the result

Together with the background and the mappings, the accepted implications are complete for the theory, but they are hard to read: conclusions are full closures, and much of what they say follows from other implications. For display, `simplified_implications` rewrites them. It uses only the *trusted* implications: the background, the confirmed implications and their images. All Sudoku experts decide every question, so every accepted implication here is confirmed, and the procedure is as follows (the companion document describes how implications accepted without a decision are treated).

1. Each premise is cut to the atoms that the rest of it does not entail. This is always safe: the simplified implication is valid and stronger.
2. Each conclusion is set to the closure of the premise minus the premise, and then cut, one atom at a time, to the atoms that do not follow from the premise by way of the background, the *other* simplified implications, and the images of all of these under the mappings. The implications are cut one after another against the others as already cut, and the images of an implication are not used in cutting it, since they would let it derive its own conclusion. Atoms are tried for removal weakest first, by the size of their own closure, so that the atom the others follow from is kept (SameCell rather than SameRow and SameColumn).
3. An implication left with an empty conclusion is redundant and is dropped.

Read together with the background and the mappings, the simplified implications entail exactly what the accepted implications do; the tests check this by rebuilding a theory from them. A premise that entails every attribute is printed as concluding *everything*, unless what is left of its conclusion entails every attribute on its own. It is not printed as $\bot$: no object has such a premise unless it has every attribute, which need not be impossible. The result is non-redundant rather than maximally readable: content that appears only after a substitution making a premise atom true is not repeated separately.

In a rule exploration, each simplified implication stands for a universal Horn sentence, a first-order rule; Sections 3 and 4 therefore call them rules. In the propositional exploration of Section 5 they are implications.

3 Solved grids

3.1 Vocabulary

Table 1 lists the predicates. The *cell equivalences* describe where cells are (SameCell, SameRow, SameColumn, SameBlock, SameBand, SameStack) and whether they hold the same value (Same); Contains(x, n) relates a cell to a digit, and SameNumber is equality of digits. Together these are the *primitive* vocabulary used by default. The predicates Peers (distinct cells sharing a unit), Apart (cells sharing no unit), Different and DifferentNumbers are available as well. Each of them is a negation or a disjunction of primitives, which Horn rules cannot define; an earlier version of the exploration that used them found 42 rules, most of which restated how they relate to each other.

3.2 Background knowledge

`sudoku_background` produces implications that hold in every grid whatever its digits; given as background, they leave the exploration to find only what the constraints of Sudoku add. For the predicates and variables present they are:

Predicate	Sorts	Declared properties
SameCell, SameRow, SameColumn, SameBlock	cell, cell	symmetric, reflexive, complemented
SameBand, SameStack, Same	cell, cell	symmetric, reflexive*
DifferentCells, DifferentRows, DifferentColumns, DifferentBlocks	cell, cell	symmetric, irreflexive, complemented
Peers, Apart, Different	cell, cell	symmetric, irreflexive*
Contains	cell, number	—
SameNumber, DifferentNumbers	number, number	symmetric, (ir)reflexive, complemented
Forced, Excluded and four Confined predicates	cell, number	— (partial grids only)
ConfinedToPairInRow, ConfinedToPairInColumn	cell, cell, number	— (partial grids only)

Table 1: The predicates. * Same and Different are complements of each other. The predicates below the line are explained in Section 4.

- the laws of equivalences (reflexivity and symmetry, which declared properties absorb, and transitivity) for the seven cell equivalences and for SameNumber;
- how the regions nest: $\mathsf{SameCell}(u,v)$ implies every cell equivalence; $\mathsf{SameRow}(u,v) \wedge \mathsf{SameColumn}(u,v) \rightarrow \mathsf{SameCell}(u,v)$; a row lies in a band and a column in a stack; SameBlock is SameBand ∧ SameStack;
- that a cell holds one value: $\mathsf{Contains}(u,n) \wedge \mathsf{Same}(u,v) \rightarrow \mathsf{Contains}(v,n)$, $\mathsf{Contains}(u,n) \wedge \mathsf{Contains}(v,n) \rightarrow \mathsf{Same}(u,v)$, and the like for SameNumber;
- contrapositive forms that Horn rules cannot derive from complements: for instance, if $\mathsf{SameRow}(u,v)$ and $\mathsf{DifferentRows}(v,w)$, then $\mathsf{DifferentRows}(u,w)$.

Every background implication is checked against the experts in the tests.

3.3 Experts

SAT. `SatSudokuRuleExpert` decides a question $P \rightarrow C$ by searching for a solved grid with an assignment that satisfies P and violates some atom of C. The grid is encoded as usual with n^3 Boolean variables $g_{r,c,d}$ (exactly one digit per cell, every digit exactly once per unit; `sudoku2sat`). Each cell variable x gets one-hot variables R^x_r and C^x_c for its row and column, and each number variable m one-hot variables Q^m_d for its digit. Atoms are defined by fresh variables with Tseitin equivalences, so that they may be used negated:

$$\mathsf{SameRow}(x,y) \equiv \bigvee_r (R^x_r \wedge R^y_r), \qquad \mathsf{SameBand}(x,y) \equiv \bigvee_b (\bigvee_{r\in b} R^x_r \wedge \bigvee_{r\in b} R^y_r),$$
$$\mathsf{SameBlock} \equiv \mathsf{SameBand} \wedge \mathsf{SameStack}, \qquad \mathsf{SameCell} \equiv \mathsf{SameRow} \wedge \mathsf{SameColumn},$$
$$V^x_d \equiv \bigvee_{r,c} (R^x_r \wedge C^x_c \wedge g_{r,c,d}) \quad (\text{cell } x \text{ holds } d),$$
$$\mathsf{Same}(x,y) \equiv \bigvee_d (V^x_d \wedge V^y_d), \qquad \mathsf{Contains}(x,m) \equiv \bigvee_d (V^x_d \wedge Q^m_d),$$

and the complements and derived predicates as negations and combinations of these. A question is one call to an incremental solver with the premise literals as assumptions and an activation literal a for the clause $\neg a \vee \bigvee_{c\in C} \neg c$. A model gives the grid and the assignment, from which all attributes are read off. The expert works for any block size and decides every question.

Z3. `Z3SudokuExpert` encodes the grid as an uninterpreted function $value(r,c)$ with range and `Distinct` constraints, cell variables as pairs of integer terms, and predicates as Z3 formulas. It agrees with the SAT expert (the tests compare the theories they produce) and is comparable in

Configuration	Grid	Atoms	Maps	Questions	Accepted	Rules	Fail	Time
`solved-pairs`	4×4	7	3	11	3	3	0	0.6 s
`solved-pairs`	9×9	7	3	14	3	3	0	0.7 s
`solved-triples`	4×4	28	107	18	3	3	0	0.8 s
`solved-quads`	4×4	42	255	30	12	9	9	5.0 s
`solved-quads`	9×9	42	255	42	3	3	0	4.1 s

Table 2: Solved-grid explorations (SAT expert, primitive vocabulary, background on). "Rules" counts the simplified rules; "fail" the accepted implications that fail on the other grid size. Times are indicative (one run on a laptop). Question counts vary slightly between runs, with the counterexamples the SAT solver happens to find; the theories do not.

speed on 4×4 grids, but it is too slow on 9×9 grids, where proving even a two-cell rule took minutes.

Checking across grid sizes. `check_rules` re-checks accepted rules with the SAT expert on grids of another block size and returns a counterexample for each rule that fails there. Since atoms are interpreted by predicate name, rules found on 4×4 grids can be checked on 9×9 grids unchanged.

3.4 Results

Table 2 summarizes the shipped configurations for solved grids.

With two cells, or three cells and two digits, the exploration finds exactly the constraints of the game, as rules:

```
Same(x, y), SameBlock(x, y) → SameCell(x, y)
Same(x, y), SameColumn(x, y) → SameCell(x, y)
Same(x, y), SameRow(x, y) → SameCell(x, y)
```

With four cells on 4×4 grids, it also finds rules about two pairs of cells with equal values, for instance

```
Same(w, y), Same(x, z), SameBlock(w, z), SameBlock(x, y), SameColumn(y, z) →
SameColumn(w, x)
```

These hold only because a 4×4 block has two columns, and the re-check on 9×9 grids marks each of them with a counterexample. The three rules stating the constraints of the game are accepted here too, but they are not printed: on 4×4 grids they follow from the nine others by identifying variables, and the simplified rules are non-redundant (Section 2.6). A reader of the 4×4 output alone would not see them. On 9×9 grids, four cells again yield only the three rules above: with these predicates and up to four cells, the rules of the game are the whole theory of solved 9×9 grids.

4 Partial grids

4.1 Objects and completion semantics

Solving techniques are about partial grids. An object of the partial-grid exploration is a partial grid G with at least one completion, together with an assignment of cells and digits to the variables. The predicates about contents speak about *all completions*:

$$G \models \mathsf{Forced}(x, n) \iff \text{every completion of } G \text{ has } n \text{ in } x,$$
$$G \models \mathsf{Excluded}(x, n) \iff \text{no completion of } G \text{ has } n \text{ in } x.$$

A rule between such atoms states that whatever is known in its premise determines what is known in its conclusion; the rules are therefore laws of deduction, valid for every partial grid. The geometric predicates are as before. Same and Contains do not apply, since a partial grid has no values of its own beyond its givens, and writing a forced digit into the grid does not change its completions.

This *completion semantics* makes knowledge complete in a way that pencil-and-paper knowledge is not, and this has a consequence for the vocabulary.

Proposition 1. *For a partial grid with at least one completion, n is excluded from every cell of x's row other than x if and only if n is forced in x. The same holds for columns and blocks, and n is forced in x if and only if every digit other than n is excluded from x.*

Proof. Every completion has n somewhere in x's row. If no completion has it in another cell of the row, every completion has it in x. Conversely, a completion with n in x has n in no other cell of the row. The other statements are proved alike. □

Predicates such as "x is the only place for n in its row" (hidden single) or "n is the only digit for x" (naked single) would therefore just restate Forced, and the library does not provide them. Hidden and naked singles appear instead as rules between Excluded and Forced atoms when the vocabulary has a variable for every cell of a unit, which is possible for 4×4 grids only.

4.2 Confinement predicates

What is not expressible with Forced and Excluded over few variables are statements about where a digit must be within a unit. All completion predicates have the form

$$G \models p(x_1, \ldots, x_k, n) \iff \text{no completion of } G \text{ has } n \text{ in a cell of } \mathrm{Out}_p(x_1, \ldots, x_k),$$

for a set Out_p of cells determined by the positions of the cell arguments:

Predicate	Out_p: the cells n is kept out of
$\mathsf{Excluded}(x, n)$	$\{x\}$
$\mathsf{Forced}(x, n)$	the cells of x's row other than x (Proposition 1)
$\mathsf{ConfinedToRowInBlock}(x, n)$	the cells of x's block outside x's row
$\mathsf{ConfinedToColumnInBlock}(x, n)$	the cells of x's block outside x's column
$\mathsf{ConfinedToBlockInRow}(x, n)$	the cells of x's row outside x's block
$\mathsf{ConfinedToBlockInColumn}(x, n)$	the cells of x's column outside x's block
$\mathsf{ConfinedToPairInRow}(x, y, n)$	the cells of x's row other than x and y
$\mathsf{ConfinedToPairInColumn}(x, y, n)$	the cells of x's column other than x and y

The first two Confined predicates are the premises of *pointing*, the next two those of *claiming* (together: locked candidates), and the pair predicates those of the *X-wing*. Their conclusions need the complements DifferentRows, DifferentColumns and DifferentBlocks ("in the same row, but another block"), which were added to the geometric vocabulary for this purpose.

A pair predicate is about x's row: $\mathsf{ConfinedToPairInRow}(x, y, n)$ with y outside that row says that n is forced in x. It is therefore symmetric in x and y only when they share the row, and the background says exactly that (an unconditional symmetry, tried first, contradicted the very first counterexample of the brute-force check of Section 6).

4.3 Background knowledge

The background of Section 3.2 is extended by what single-valuedness and equality give for the completion predicates: $\mathsf{Forced}(u, n) \wedge \mathsf{Excluded}(u, n) \to M$; two digits forced in one cell are equal; a forced digit excludes every other digit of its cell; a cell where a digit is forced differs

from every cell where it is excluded; equal cells and equal digits are interchangeable in every completion predicate; $\mathsf{ConfinedToPairInRow}(u, u, n)$ is equivalent to $\mathsf{Forced}(u, n)$. Rules that are techniques, such as the naked single, are deliberately left out of the background.

Geometry first. With several cell variables and all geometric predicates, many accepted rules are facts about where cells can be, such as "if w and x share a block and ..., then y and z share a block". With `geometry_first`, the geometric predicates are first explored alone over the same cell variables (`geometry_basis`, with the solved-grid SAT expert, since geometry does not depend on digits), and the accepted rules are added to the background. The final theory is the same; only rules involving completion predicates are reported. In the four-cell configuration this removes 50 of 131 simplified rules.

4.4 The enumerating expert

For 4×4 grids, `PartialGridExpert` lists the 288 solved grids $s_1, \ldots, s_{288}$ in advance. The givens are Boolean variables $g_{r,c,d}$ (at most one per cell). Completion is expressed by

$$cons_i \equiv \bigwedge_{r,c} \bigwedge_{d \neq s_i(r,c)} \neg g_{r,c,d} \qquad (s_i \text{ is a completion of the givens}),$$

together with the clause $\bigvee_i cons_i$ (some completion exists). A completion predicate becomes a finite conjunction over the listed grids:

$$p(\vec{x}, n) \equiv \bigwedge_i (\neg cons_i \vee in_i(p, \vec{x}, n)),$$

where $in_i(p, \vec{x}, n)$ says that s_i keeps the digit of n out of $\mathrm{Out}_p(\vec{x})$. Since s_i is fixed, this is a disjunction over the possible positions of the cell variables and digits for which s_i satisfies the condition, for instance $in_i(\mathsf{Excluded}, x, n) \equiv \neg \bigvee_d (Q^n_d \wedge \bigvee_{(r,c):s_i(r,c)=d} (R^x_r \wedge C^x_c))$. For the pair predicates, the cell of x's row holding the digit in s_i is determined by the row and the digit, which keeps the encoding small.

4.5 The CEGAR expert

For 9×9 grids, listing the solved grids is out of the question. A counterexample to $P \to C$ is a partial grid G with an assignment α such that every completion satisfies the completion atoms of P and some completion violates an atom of C (or a geometric atom of C fails). Writing $\mathrm{comp}(G)$ for the completions, the question is

$$\exists G, \alpha, W \in \mathrm{comp}(G) : \; W \not\models \bigwedge C \;\wedge\; \forall S \in \mathrm{comp}(G) : \; S \models \bigwedge P,$$

where $S \models p(\vec{x}, n)$ means that the solved grid S keeps the digit of n out of $\mathrm{Out}_p(\vec{x})$. This is an $\exists\forall$ problem. `CegarPartialGridExpert` decides it by counterexample-guided refinement [2] with two SAT solvers (Algorithm 1).

The guessing solver has the givens $g_{r,c,d}$, a full Sudoku encoding of W with $g_{r,c,d} \to w_{r,c,d}$, the position and digit variables of the variables, and, for every $S \in \mathcal{S}$, a literal $cons_S$ as above. Satisfaction of a completion atom by W is encoded symbolically (for instance, $\mathsf{ConfinedToRowInBlock}(x, n)$ holds in W iff no cell of x's block outside x's row has the digit of n in W), and satisfaction by $S \in \mathcal{S}$ as in the enumerating expert. The constraints of a question are guarded by an activation literal, so that one incremental solver serves all questions. The verifying solver holds a plain Sudoku encoding; each verification is a call with the givens and one cell literal as assumptions.

Proposition 2. *Algorithm 1 terminates, and its answers are correct.*

Algorithm 1 Deciding $P \to C$ for partial grids (`CegarPartialGridExpert.validate`)

$\mathcal{S} \leftarrow$ the completions found for earlier questions ▷ kept across questions
loop
  guess: find givens G, an assignment α and a solved grid W such that
   W completes G, W satisfies P and violates C (under α), and
   every $S \in \mathcal{S}$ that completes G satisfies the completion atoms of P
  if there is none **then return** "$P \to C$ holds"
  end if
  verify: for each completion atom $p(\vec{x}, n)$ of P and each cell $b \in \mathrm{Out}_p(\alpha\vec{x})$,
   look for a completion of G with $\alpha(n)$ in b
  if a completion S is found **then**
    $\mathcal{S} \leftarrow \mathcal{S} \cup \{S\}$ ▷ refine
  else
    return the counterexample (G, α), its row computed exactly by the verifier
  end if
end loop

Configuration	Grid	Atoms	Maps	Questions	Accepted	Rules	Time
`partial-cell`	4×4	20	255	6	2	2	0.5 s
`partial-row`	4×4	26	255	22	6	5	2.5 s
`partial-column`	4×4	26	255	22	6	5	1.7 s
`partial-block`	4×4	26	255	23	6	5	1.9 s
`partial-units`	4×4	21	26	53	16	16	1.2 s
`partial-full`	4×4	38	255	189	84	81	123.8 s
`partial-locked`	4×4	20	3	69	46	44	30.0 s
`partial9-locked`	9×9	20	3	69	32	30	211.9 s

Table 3: Partial-grid explorations. Times are indicative (one run on a laptop). Question counts vary slightly between runs, with the counterexamples the SAT solver happens to find; the theories do not.

Proof. A counterexample returned has passed the verification, so every completion of G satisfies the completion atoms of P; W is a completion violating C, and the geometric atoms are those of α. If a counterexample (G^*, α^*) with a violating completion W^* exists, then (G^*, α^*, W^*) satisfies every constraint of the guess, since all completions of G^*, in particular those in $\mathcal{S}$, satisfy P; so the guess cannot fail. Each refinement adds a completion S of the current G that violates P; the guess is then excluded, so $S \notin \mathcal{S}$ before, and since there are finitely many solved grids, the loop terminates. □

In the runs reported here the loop needed between two and eight guesses per question. On 4×4 grids the CEGAR expert reaches the same theories as the enumerating one and is several times faster.

4.6 Results

Table 3 summarizes the shipped configurations for partial grids.

Singles and elimination (4×4). With one cell and four digits (`partial-cell`) the exploration finds the naked single in Horn form:

`DifferentNumbers(n1, n2)`, `...`, `DifferentNumbers(n3, n4)`, `Excluded(x, n2)`, `Excluded(x, n3)`, `Excluded(x, n4)` → `Forced(x, n1)`

With the four cells of a row, column or block, it finds the hidden single (a digit excluded from three distinct cells of a unit is forced in the fourth), elimination, and that a digit has at most one forced place in a unit:

```
DifferentCells(y, z), Forced(z, n), SameRow(y, z) → Excluded(y, n)
Forced(y, n), Forced(z, n), SameRow(y, z) → SameCell(y, z)
```

The full four-cell vocabulary (`partial-full`, explored geometry first) yields 81 rules, listed in Appendix A: these techniques, interactions of blocks and lines that need a fourth cell, and many rules in contrapositive Horn form.

Locked candidates (9×9). With two cells, one digit, the geometry and the completion predicates (`partial9-locked`, CEGAR expert), the exploration finds pointing directly,

```
ConfinedToRowInBlock(y, n), DifferentBlocks(x, y), SameRow(x, y) →
Excluded(x, n)
```

and likewise for columns. Claiming follows from two rules,

```
ConfinedToBlockInRow(y, n) → ConfinedToRowInBlock(y, n)
ConfinedToRowInBlock(y, n), DifferentRows(x, y), SameBlock(x, y) →
Excluded(x, n)
```

the first of which holds because a digit occurs once per block: if its place within a row lies in a block, that place is its cell in the block. Also found: confinement to x's row and to x's column within x's block forces the digit in x. Appendix B lists all 30 rules.

The X-wing (9×9). The X-wing can be stated with five cell variables and the pair predicates:

```
ConfinedToPairInRow(a, b, n), ConfinedToPairInRow(c, d, n), SameColumn(a, c),
SameColumn(b, d), DifferentRows(a, c), SameColumn(a, e), DifferentRows(a, e),
DifferentRows(c, e) → Excluded(e, n)
```

Asked directly, the CEGAR expert confirms it on 9×9 grids in seconds and refutes, with concrete counterexamples, the variants without $\mathsf{DifferentRows}(a, c)$, without $\mathsf{SameColumn}(b, d)$, and without the second pair. An exploration that would discover it, however, does not finish: see Section 8.

5 Propositional exploration

The propositional exploration of 4×4 grids has the 64 attributes (r, c, d), "cell (r, c) holds d"; its objects are solved grids, and `SudokuExpert` answers with a SAT solver, returning a solved grid that contains the premise and misses some conclusion attribute.

Background. The constraints of the game are given as background (`get_sudoku_background_implications`): two digits in a cell, or one digit twice in a unit, imply every attribute; and when the other cells of a unit hold the other digits, the remaining cell holds the remaining digit. There are $(n-1)!$ premises of the second kind per cell, digit and unit, 1472 background implications in all for $n = 4$, which is practical for 4×4 grids only.

Symmetries. The symmetry group of Sudoku grids is generated by permutations of the bands, of the rows within each band, of the stacks and of the columns within each stack, transposition, and relabelling of the digits. Since the general loop maps by each mapping once and never by compositions (Section 2.3), the mappings must be the whole group rather than generators. For 4×4 grids it has $8 \cdot 8 \cdot 2 \cdot 24 = 3072$ elements; for 9×9 grids $(3!^4)^2 \cdot 2 \cdot 9! \approx 1.2 \cdot 10^{12}$, which is why only block size 2 is supported. The background is a union of orbits, so it is mapped once per orbit (Section 2.5).

Results. The exploration asks 6 questions, sees all 288 solved grids, and accepts 4 implications beyond the background, each standing for its images under the group:

```
(2, 3, 4), (3, 1, 3) → (3, 0, 4)
(1, 3, 4), (3, 1, 4) → (2, 2, 4)
(1, 3, 4), (3, 0, 3), (3, 1, 2) → (3, 3, 1)
(1, 1, 4), (1, 3, 3), (2, 3, 2), (3, 1, 2), (3, 2, 3) → (3, 3, 1)
```

They are deductions that go beyond the background: in the first, row 3 needs a 4; the bottom-right block has its 4 at $(2, 3)$, so the 4 of row 3 is not in that block, and $(3, 1)$ holds a 3, so it must be at $(3, 0)$ (rows and columns are numbered from 0). Nearly all of the running time goes into enumerating the closed sets, the partial grids from which nothing more follows, rather than into questions.

6 How the results were checked

The experts are programs, and an error in an encoding would silently produce a wrong theory. The test suite (`tests/`) checks the experts and the results in several independent ways.

- **Brute force on 4×4 grids.** For partial grids, every counterexample an expert returns is re-evaluated against the 288 solved grids directly: the premise must hold, the conclusion must fail, and every attribute must have the value the expert claims. Accepted rules are tested on random partial grids. Declared properties of predicates are checked on solved grids of both sizes, and background implications against the experts.
- **Agreement of experts.** The SAT and Z3 experts for solved grids, and the enumerating and CEGAR experts for partial grids, must produce identical theories on the same vocabulary.
- **Across sizes.** Rules found on 4×4 grids are re-checked on 9×9 grids; for the propositional exploration, the accepted implications and all their images under the group are checked against all 288 grids.
- **The simplified implications.** A theory is rebuilt from them together with the background and the mappings, and must entail and be entailed by the accepted implications.
- **Mutation checks.** The tests were run against deliberately broken code (Forced and Excluded swapped in an encoding, wrong confinement cells, a missing update of the transposed rows, a wrong closure condition, an implication allowed to cut itself with its own images) to make sure they fail.

On 9×9 grids no independent check of the partial-grid results exists. The evidence is that the CEGAR expert reproduces the 4×4 results of the enumerating expert exactly, with the same code.

7 User's guide

7.1 Running an exploration

Explorations are described by TOML files. The runner takes one or several, by path or by the name of a configuration shipped in `explorations/sudoku/configs`:

```
python explorations/sudoku/explore.py partial-row
python explorations/sudoku/explore.py my-exploration.toml --block-size 3
python explorations/sudoku/explore.py partial9-locked --output results/locked.txt
```

Name	Content
`solved-pairs`	two cells, cell equivalences
`solved-triples`	three cells, two digits, primitive vocabulary
`solved-quads`	four cells, cell equivalences
`partial-cell`	one cell, four digits: naked singles
`partial-row`, `-column`, `-block`	the four cells of a unit: hidden singles, elimination
`partial-units`	three cells, all geometry: interaction of units
`partial-full`	four cells, all geometry, geometry first
`partial-locked`, `partial9-locked`	two cells with the Confined predicates, 4×4 and 9×9
`propositional`	cell assignments of the 4×4 grid

Table 4: Shipped configurations.

`--block-size` and `--no-background` override the configuration; `--output` also writes the result to a file, preceded by the configuration that produced them. All configurations are checked before any of them runs, and each run prints its numbers of atoms and mappings first, since these decide how long it takes. The project requires Python 3.11 (for `tomllib`), `python-sat`, and `z3-solver` for the Z3 expert.

7.2 Configuration files

```
grid = "partial"          # "solved", "partial" or "propositional"
block_size = 2            # 3 for 9x9 grids
cells = ["w", "x", "y", "z"]
numbers = ["n"]
predicates = ["SameCell", "DifferentCells", "SameRow", "Forced", "Excluded"]
background = true         # what holds in any grid
geometry_first = false    # explore the geometry first, add it to the background
expert = "sat"            # "z3" for solved grids, "cegar" for partial ones
check_block_size = 3      # solved grids: re-check rules there; 0 skips
```

A propositional configuration takes only `block_size`, `background` and `symmetries`. Settings that cannot run are reported with a reason: unknown keys, unknown predicates, predicates that do not apply to the kind of grid, a predicate without a variable of a sort it needs, partial grids of block size 3 with the enumerating expert, and so on. The shipped configurations are listed in Table 4.

7.3 From Python

`sudoku_rule_exploration` builds any rule exploration with the same checks; `load_config` reads a configuration, and `simplified_implications` prints the result:

```
from explorations.sudoku import load_config, sudoku_rule_exploration
from conceptual_exploration import simplified_implications

exploration = sudoku_rule_exploration(
    "partial", 3, cells=["x", "y"], numbers=["n"],
    predicates=["SameRow", "DifferentBlocks", "Excluded", "ConfinedToRowInBlock"],
    expert="cegar")
exploration.run()
for rule in simplified_implications(exploration.base):
    print(rule)
```

7.4 Where the code is

`src/conceptual_exploration` the general library: `logic/` (atoms, predicates with properties, variables, variable maps), `core/` (bit sets, implication theories, truth values), `exploration/` (the exploration base, the loop, rule exploration, rounds, the simplified implications).

`explorations/sudoku/sudoku.py` the domain: SAT encoding of grids, the propositional expert and symmetries, the Z3 expert, the predicate tables, the background, the SAT experts for solved and partial grids, the CEGAR expert, and `sudoku_rule_exploration`.

`explorations/sudoku/config.py`, `explore.py`, `configs/` configurations and the runner.

8 Limits met so far

The number of closed sets. The exploration enumerates every set closed under the current theory, and in most explorations here this, not the expert, takes nearly all the time. The X-wing vocabulary over five cells has about 50 atoms and 3124 substitution maps; in a run of half an hour, the exploration asked about 45 questions, and profiling showed NextClosure taking hundreds of thousands of steps per question while the CEGAR expert used a few percent of the time. The propositional exploration of 4×4 grids asks only 6 questions and spends its time in the same way. Faster closures (Section 2.5) moved this limit but cannot remove it.

Horn form. Rules are universal Horn sentences over positive atoms. Many interesting statements are disjunctive or negative, and complements recover only part of them. A consequence is that many printed rules are contrapositive forms of simpler ones, for instance "if y and z are distinct cells with the same forced digit and w, x lie in their rows, then w and x are distinct", a restatement of the fact that a digit is forced at most once per unit. They are correct but make long outputs hard to read.

Variables per unit. A technique that speaks about a whole unit needs a variable for each of its cells unless a predicate quantifies over the unit. For 4×4 grids four variables suffice and hidden singles appear as rules; for 9×9 grids they would need nine. Under completion semantics, predicates quantifying over a unit coincide with Forced (Proposition 1), so the techniques that become accessible on 9×9 grids are those with genuinely new premises, such as confinement.

Artefacts of small grids. On 4×4 grids, bands and blocks have two rows, and many rules hold for that reason only (Section 3.4); confinement to a block within a row then even implies confinement to a row within the block, so claiming collapses into pointing. Rules found on 4×4 grids should be re-checked on 9×9 grids, which `check_rules` does for solved grids. For partial grids no such check is implemented yet.

9 Directions for further development

Exploring around a fixed configuration. The X-wing, swordfish and similar techniques have a fixed geometric configuration. Instead of discovering the configuration together with the technique, the expert could be restricted to placements of the variables in that configuration, and only the completion atoms explored, with the symmetries of the configuration (for the X-wing, exchanging the two rows or the two columns where this preserves the other relations) as mappings in place of substitutions. This would reduce the X-wing vocabulary from about 50 to about 30 atoms with strong structure. Configurations could also be enumerated systematically, up to symmetry, for a given number of cells.

Targeted exploration. Often only rules with a particular conclusion are wanted, say those concluding $\mathsf{Excluded}(e, n)$. Rather than the whole canonical basis, the exploration could compute the minimal premises for a given conclusion atom, or explore only premises from a chosen set of atoms. Both avoid enumerating the closed sets of the full vocabulary, which is what makes larger vocabularies infeasible now.

Partial symmetry of arguments. $\mathsf{ConfinedToPairInRow}(x, y, n)$ is symmetric in x and y when they share the row. The library supports declared symmetry only for binary predicates; declaring symmetry of selected argument positions, possibly under a condition, would remove duplicate atoms from vocabularies with pair predicates.

Other semantics of knowledge. Completion semantics describes what follows, not how it is found. An alternative object model would be the states of a solver: grids with candidate sets, closed under a fixed set of deduction steps. Rules would then describe what a given repertoire of techniques derives, and exploring with and without a technique would show what it adds. A related variant restricts objects to puzzles with a unique solution, which is the setting of uniqueness-based techniques (unique rectangles); under completion semantics with a unique completion every cell is forced, so these techniques need the restriction to be stated as background or as part of the predicates rather than of the objects.

Grids of other sizes and shapes. Rules about solved grids could be checked for 16×16 grids as well, and rules holding for all block sizes separated from those of particular sizes. Variants of Sudoku change the units: diagonal Sudoku adds two units, Jigsaw Sudoku has irregular blocks, Killer Sudoku adds cages with sums. The experts generalize directly, since the units enter only through the SAT encoding; the predicates and the symmetry groups have to be adapted.

Larger theories. For the propositional exploration, the remaining cost is the enumeration of closed sets. Exploring a part of the grid (one band, or a row and a block), using canonical representatives of orbits instead of storing all images, or computing closures with compiled code would extend what is feasible.

Using the rules. The rules found are inference rules for Sudoku. They could be turned into a solver that applies them, with each step justified by a rule; exploring which rules are needed for given puzzles would then relate the exploration to the grading of puzzles by difficulty.

Readability. The simplified rules are non-redundant, which makes some of their content appear only through substitutions or contrapositives. Optional post-processing could group rules by the configuration of their cells, print the images that shorten premises, or recognize contrapositive forms and print them together with the rule they restate.

References

[1] B. Ganter and S. Obiedkov. *Conceptual Exploration*. Springer, 2016.

[2] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification (CAV 2000)*, LNCS 1855, Springer, 2000.

A Rules of the four-cell partial-grid exploration

Configuration `partial-full` (4×4 grids, four cells, all geometry, explored geometry first).

[1] Forced(y, n), Forced(z, n), SameBlock(y, z) → SameCell(y, z)

[2] Forced(y, n), Forced(z, n), SameColumn(y, z) → SameCell(y, z)

[3] Forced(y, n), Forced(z, n), SameRow(y, z) → SameCell(y, z)

[4] DifferentCells(y, z), Forced(z, n), SameBlock(y, z) → Excluded(y, n)

[5] DifferentCells(y, z), Forced(z, n), SameColumn(y, z) → Excluded(y, n)

[6] DifferentCells(y, z), Excluded(y, n), Excluded(z, n), SameBlock(x, z), SameBlock(y, z), SameColumn(x, z), SameColumn(y, z) → Excluded(x, n)

[7] DifferentCells(y, z), Forced(z, n), SameRow(y, z) → Excluded(y, n)

[8] DifferentCells(y, z), Forced(w, n), Forced(y, n), SameBlock(x, y), SameColumn(w, z), SameRow(x, z) → DifferentCells(w, x)

[9] DifferentCells(y, z), Forced(w, n), Forced(x, n), SameBlock(x, y), SameColumn(w, z), SameRow(x, z) → DifferentCells(w, y)

[10] DifferentCells(y, z), Forced(x, n), Forced(y, n), SameBlock(w, y), SameColumn(w, z), SameRow(x, z) → DifferentCells(w, x)

[11] DifferentCells(y, z), Forced(w, n), Forced(x, n), SameBlock(w, y), SameColumn(w, z), SameRow(x, z) → DifferentCells(x, y)

[12] DifferentCells(y, z), Forced(x, n), Forced(y, n), SameBlock(w, z), SameBlock(x, z), SameColumn(w, z), SameRow(x, z) → DifferentCells(w, y)

[13] DifferentCells(y, z), Forced(w, n), Forced(y, n), SameBlock(w, z), SameBlock(x, z), SameColumn(w, z), SameRow(x, z) → DifferentCells(x, y)

[14] DifferentCells(y, z), Forced(w, n), Forced(y, n), SameColumn(w, z), SameColumn(x, y), SameRow(x, z) → DifferentCells(w, x)

[15] DifferentCells(y, z), Forced(w, n), Forced(x, n), SameColumn(w, z), SameColumn(x, y), SameRow(x, z) → DifferentCells(w, y)

[16] DifferentCells(y, z), Forced(x, n), Forced(y, n), SameBlock(w, y), SameBlock(x, z), SameColumn(w, y), SameRow(x, z) → DifferentCells(w, z)

[17] DifferentCells(y, z), Forced(w, n), Forced(z, n), SameBlock(w, y), SameBlock(x, z), SameColumn(w, y), SameRow(x, z) → DifferentCells(x, y)

[18] DifferentCells(y, z), Forced(w, n), Forced(y, n), SameBlock(x, y), SameColumn(w, x), SameRow(x, z) → DifferentCells(w, z)

[19] DifferentCells(y, z), Forced(x, n), Forced(y, n), SameBlock(w, y), SameColumn(w, x), SameRow(x, z) → DifferentCells(w, z)

[20] DifferentCells(y, z), Forced(w, n), Forced(z, n), SameBlock(w, y), SameBlock(x, z), SameColumn(w, x), SameRow(x, z) → DifferentCells(x, y)

[21] DifferentCells(y, z), Excluded(y, n), Excluded(z, n), SameBlock(x, z), SameBlock(y, z), SameRow(x, z), SameRow(y, z) → Excluded(x, n)

[22] DifferentCells(y, z), Forced(w, n), Forced(y, n), SameColumn(x, z), SameRow(w, z), SameRow(x, y) → DifferentCells(w, x)

[23] DifferentCells(y, z), Forced(w, n), Forced(x, n), SameColumn(x, z), SameRow(w, z), SameRow(x, y) → DifferentCells(w, y)

[24] DifferentCells(y, z), Forced(x, n), Forced(z, n), SameColumn(w, x), SameRow(w, z), SameRow(x, y) → DifferentCells(w, y)

[25] DifferentCells(y, z), Forced(w, n), Forced(y, n), SameBlock(x, y), SameColumn(x, z), SameRow(w, x) → DifferentCells(w, z)

[26] DifferentCells(y, z), Forced(x, n), Forced(y, n), SameBlock(w, y), SameColumn(x, z), SameRow(w, x) → DifferentCells(w, z)

[27] DifferentCells(y, z), Forced(w, n), Forced(z, n), SameBlock(w, y), SameBlock(x, z), SameColumn(x, z), SameRow(w, x) → DifferentCells(x, y)

[28] DifferentCells(y, z), Forced(x, n), Forced(z, n), SameColumn(w, z), SameColumn(x, y), SameRow(w, x) → DifferentCells(w, y)

[29] DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, z), SameBlock(y, z), SameColumn(w, x), SameRow(y, z) → Excluded(w, n)

[30] DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, z), SameBlock(y, z), SameColumn(x, y), SameRow(w, z) → Excluded(w, n)

[31] DifferentCells(w, z), DifferentCells(x, z), DifferentCells(y, z), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, z), SameBlock(y, z), SameColumn(w, z), SameColumn(x, y) → SameCell(x, y)

[32] DifferentCells(w, z), DifferentCells(x, z), DifferentCells(y, z), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, y), SameColumn(w, z), SameColumn(x, z), SameColumn(y, z) → SameCell(x, y)

[33] DifferentCells(w, z), Excluded(x, n), Excluded(y, n), Forced(z, n), SameBlock(w, y), SameBlock(x, z), SameColumn(w, y), SameColumn(x, z), SameRow(x, y) → Excluded(w, n)

[34] DifferentCells(w, z), DifferentCells(x, z), DifferentCells(y, z), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, z), SameBlock(y, z), SameRow(w, z), SameRow(x, y) → SameCell(x, y)

[35] DifferentCells(x, z), Excluded(w, n), Excluded(y, n), Forced(z, n), SameBlock(w, z), SameBlock(x, y), SameColumn(w, y), SameRow(w, z), SameRow(x, y) → Excluded(x, n)

[36] DifferentCells(w, z), DifferentCells(x, z), DifferentCells(y, z), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, y), SameRow(w, z), SameRow(x, z), SameRow(y, z) → SameCell(x, y)

[37] DifferentCells(w, z), DifferentCells(x, y), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, z), SameBlock(y, z), SameColumn(w, z), SameColumn(x, y) → SameColumn(y, z)

[38] DifferentCells(w, z), DifferentCells(x, y), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, y), SameColumn(w, z), SameColumn(x, z), SameColumn(y, z) → SameBlock(y, z)

[39] DifferentCells(w, z), DifferentCells(x, y), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, z), SameBlock(y, z), SameRow(w, z), SameRow(x, y) → SameRow(y, z)

[40] DifferentCells(w, z), DifferentCells(x, y), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, y), SameRow(w, z), SameRow(x, z), SameRow(y, z) → SameBlock(y, z)

[41] DifferentCells(y, z), Forced(y, n), Forced(z, n), SameBlock(w, y), SameBlock(x, z) → DifferentCells(w, x)

[42] DifferentCells(y, z), Forced(y, n), Forced(z, n), SameColumn(w, y), SameColumn(x, z) → DifferentCells(w, x)

[43] DifferentCells(w, z), DifferentCells(x, y), Excluded(y, n), Forced(z, n), SameBlock(w, y), SameBlock(x, y), SameColumn(w, y), SameColumn(x, y), SameRow(x, z) → Excluded(w, n)

[44] DifferentCells(y, z), Forced(y, n), Forced(z, n), SameRow(w, y), SameRow(x, z) → DifferentCells(w, x)

[45] DifferentCells(w, z), DifferentCells(x, y), Excluded(z, n), Forced(y, n), SameBlock(w, z), SameBlock(x, z), SameColumn(w, y), SameRow(w, z), SameRow(x, z) → Excluded(x, n)

[46] DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, z), SameBlock(y, z), SameColumn(x, y) → SameColumn(w, y)

[47] DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(x, y), SameColumn(w, z), SameColumn(x, z), SameColumn(y, z) → SameBlock(w, y)

[48] DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(x, n), Excluded(y, n), SameBlock(w, z), SameBlock(x, z), SameBlock(y, z), SameColumn(w, x), SameRow(y, z) → Excluded(w, n)

[49] DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, z), SameBlock(y, z), SameRow(x, y) → SameRow(w, y)

[50] DifferentCells(x, y), Excluded(w, n), Excluded(x, n), Excluded(y, n), Forced(z, n), SameBlock(w, z), SameBlock(x, y), SameColumn(w, y), SameColumn(x, y), SameRow(w, z) → SameBlock(y, z)

[51] DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(x, n), Excluded(y, n), SameBlock(w, z), SameBlock(x, z), SameBlock(y, z), SameColumn(x, z), SameRow(w, y) → Excluded(w, n)

[52] DifferentCells(x, y), Excluded(w, n), Excluded(x, n), Excluded(y, n), Forced(z, n), SameBlock(w, z), SameBlock(x, y), SameColumn(w, z), SameRow(w, y), SameRow(x, y) → SameBlock(y, z)

[53] DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(x, y), SameRow(w, z), SameRow(x, z), SameRow(y, z) → SameBlock(w, y)

[54] DifferentCells(w, y), DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, z), SameBlock(y, z) → SameCell(w, x)

[55] DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(y, n), Excluded(z, n), Forced(w, n), SameBlock(w, x), SameBlock(y, z), SameColumn(x, z), SameColumn(y, z) → SameColumn(w, z)

[56] DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(y, n), Excluded(z, n), Forced(w, n), SameBlock(x, z), SameBlock(y, z), SameColumn(w, x), SameColumn(y, z) → SameBlock(w, z)

[57] DifferentCells(w, y), DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameColumn(w, z), SameColumn(x, z), SameColumn(y, z) → SameCell(w, x)

[58] DifferentCells(w, y), Excluded(z, n), Forced(w, n), Forced(y, n), SameBlock(w, x), SameBlock(y, z), SameColumn(x, z), SameRow(y, z) → SameColumn(w, z)

[59] DifferentCells(w, y), DifferentCells(x, z), Forced(w, n), Forced(y, n), SameBlock(w, z), SameBlock(x, z), SameColumn(x, z), SameRow(y, z) → SameRow(w, x)

[60] DifferentCells(w, z), DifferentCells(x, z), Excluded(w, n), Excluded(x, n), Excluded(z, n), Forced(y, n), SameBlock(w, x), SameBlock(y, z), SameColumn(w, z), SameColumn(x, z), SameRow(y, z) → SameCell(w, x)

[61] DifferentCells(w, y), Excluded(z, n), Forced(w, n), Forced(y, n), SameBlock(w, x), SameBlock(y, z), SameColumn(y, z), SameRow(x, z) → SameRow(w, z)

[62] DifferentCells(w, y), DifferentCells(x, z), Forced(w, n), Forced(y, n), SameBlock(w, z), SameBlock(x, z), SameColumn(y, z), SameRow(x, z) → SameColumn(w, x)

[63] DifferentCells(y, z), Excluded(x, n), Forced(y, n), Forced(z, n), SameBlock(w, z), SameBlock(x, z), SameColumn(w, y), SameRow(x, z) → SameColumn(x, y)

[64] DifferentCells(y, z), Excluded(w, n), Forced(y, n), Forced(z, n), SameBlock(w, y), SameBlock(x, y), SameColumn(w, y), SameRow(x, z) → SameRow(w, z)

[65] DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(y, n), Excluded(z, n), Forced(w, n), SameBlock(w, x), SameBlock(y, z), SameRow(x, z), SameRow(y, z) → SameRow(w, z)

[66] DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, x), SameBlock(y, z), SameColumn(w, x), SameRow(x, z), SameRow(y, z) → Excluded(w, n)

[67] DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, x), SameBlock(y, z), SameColumn(x, z), SameColumn(y, z), SameRow(w, x) → Excluded(w, n)

[68] DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(y, n), Excluded(z, n), Forced(w, n), SameBlock(x, z), SameBlock(y, z), SameRow(w, x), SameRow(y, z) → SameBlock(w, z)

[69] DifferentCells(w, z), DifferentCells(x, z), Excluded(w, n), Excluded(x, n), Excluded(z, n), Forced(y, n), SameBlock(w, x), SameBlock(y, z), SameColumn(y, z), SameRow(w, z), SameRow(x, z) → SameCell(w, x)

[70] DifferentCells(w, y), DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(w, n), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameRow(w, z), SameRow(x, z), SameRow(y, z) → SameCell(w, x)

[71] DifferentCells(w, x), DifferentCells(w, y), DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameBlock(w, z), SameBlock(x, z), SameBlock(y, z) → Forced(w, n)

[72] DifferentCells(w, x), DifferentCells(w, y), DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameColumn(w, z), SameColumn(x, z), SameColumn(y, z) → Forced(w, n)

[73] DifferentCells(w, y), DifferentCells(w, z), DifferentCells(y, z), Excluded(w, n), Excluded(y, n), Excluded(z, n), Forced(x, n), SameBlock(w, y), SameBlock(x, z), SameColumn(x, z), SameRow(y, z) → SameRow(w, x)

[74] DifferentCells(w, z), Excluded(x, n), Forced(w, n), Forced(z, n), SameBlock(w, y), SameBlock(x, z), SameColumn(x, z), SameRow(y, z) → SameRow(w, x)

[75] DifferentCells(w, x), DifferentCells(w, z), DifferentCells(x, z), Excluded(w, n), Excluded(x, n), Excluded(z, n), Forced(y, n), SameBlock(w, x), SameBlock(y, z), SameColumn(x, z), SameRow(y, z) → SameColumn(w, y)

[76] DifferentCells(w, z), Excluded(y, n), Forced(w, n), Forced(z, n), SameBlock(w, x), SameBlock(y, z), SameColumn(x, z), SameRow(y, z) → SameColumn(w, y)

[77] DifferentCells(w, x), DifferentCells(w, z), DifferentCells(x, z), DifferentCells(y, z), Excluded(w, n), Excluded(x, n), SameBlock(w, x), SameBlock(y, z), SameColumn(w, z), SameColumn(x, z), SameRow(y, z) → Excluded(y, n)

[78] DifferentCells(w, x), DifferentCells(w, z), DifferentCells(x, z), Excluded(x, n), Excluded(z, n), Forced(y, n), SameBlock(w, x), SameBlock(y, z), SameColumn(w, z), SameColumn(x, z), SameRow(y, z) → Forced(w, n)

[79] DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(x, n), Excluded(y, n), SameBlock(w, z), SameBlock(x, y), SameColumn(w, z), SameRow(x, z), SameRow(y, z) → Excluded(w, n)

[80] DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(y, n), Excluded(z, n), Forced(w, n), SameBlock(w, z), SameBlock(x, y), SameColumn(w, z), SameRow(x, z), SameRow(y, z) → Forced(x, n)

[81] DifferentCells(w, x), DifferentCells(w, y), DifferentCells(w, z), DifferentCells(x, y), DifferentCells(x, z), DifferentCells(y, z), Excluded(x, n), Excluded(y, n), Excluded(z, n), SameRow(w, z), SameRow(x, z), SameRow(y, z) → Forced(w, n)

B Rules of locked candidates on 9×9 grids

Configuration `partial9-locked`.

[1] `ConfinedToBlockInColumn(y, n)` → `ConfinedToColumnInBlock(y, n)`

[2] `ConfinedToBlockInRow(y, n)` → `ConfinedToRowInBlock(y, n)`

[3] `ConfinedToColumnInBlock(y, n)`, `ConfinedToRowInBlock(y, n)` → `Forced(y, n)`

[4] `ConfinedToColumnInBlock(x, n)`, `ConfinedToRowInBlock(y, n)`, `Excluded(y, n)` → `DifferentCells(x, y)`

[5] `ConfinedToColumnInBlock(y, n)`, `ConfinedToRowInBlock(x, n)`, `Excluded(y, n)` → `DifferentCells(x, y)`

[6] `Forced(y, n)` → `ConfinedToBlockInColumn(y, n)`, `ConfinedToBlockInRow(y, n)`

[7] `ConfinedToColumnInBlock(x, n)`, `ConfinedToColumnInBlock(y, n)`, `SameBlock(x, y)` → `SameColumn(x, y)`

[8] `ConfinedToRowInBlock(x, n)`, `ConfinedToRowInBlock(y, n)`, `SameBlock(x, y)` → `SameRow(x, y)`

[9] `ConfinedToColumnInBlock(x, n)`, `ConfinedToColumnInBlock(y, n)`, `SameColumn(x, y)` → `SameBlock(x, y)`

[10] `ConfinedToColumnInBlock(y, n)`, `SameBlock(x, y)`, `SameColumn(x, y)` → `ConfinedToBlockInColumn(x, n)`

[11] `ConfinedToRowInBlock(y, n)`, `Excluded(y, n)`, `SameBlock(x, y)`, `SameColumn(x, y)` → `Excluded(x, n)`

[12] `ConfinedToRowInBlock(x, n)`, `ConfinedToRowInBlock(y, n)`, `SameRow(x, y)` → `SameBlock(x, y)`

[13] `ConfinedToRowInBlock(y, n)`, `SameBlock(x, y)`, `SameRow(x, y)` → `ConfinedToBlockInRow(x, n)`

[14] `ConfinedToColumnInBlock(y, n)`, `Excluded(y, n)`, `SameBlock(x, y)`, `SameRow(x, y)` → `Excluded(x, n)`

[15] `DifferentCells(x, y)`, `Forced(x, n)`, `Forced(y, n)` → `DifferentBlocks(x, y)`

[16] `ConfinedToColumnInBlock(x, n)`, `ConfinedToColumnInBlock(y, n)`, `DifferentBlocks(x, y)` → `DifferentColumns(x, y)`

[17] `ConfinedToRowInBlock(x, n)`, `ConfinedToRowInBlock(y, n)`, `DifferentBlocks(x, y)` → `DifferentRows(x, y)`

[18] `ConfinedToColumnInBlock(x, n)`, `ConfinedToRowInBlock(y, n)`, `Excluded(y, n)`, `SameBlock(x, y)` → `DifferentColumns(x, y)`

[19] `ConfinedToColumnInBlock(y, n)`, `ConfinedToRowInBlock(x, n)`, `Excluded(y, n)`, `SameBlock(x, y)` → `DifferentRows(x, y)`

[20] `DifferentCells(x, y)`, `Forced(y, n)`, `SameBlock(x, y)` → `Excluded(x, n)`

[21] `ConfinedToColumnInBlock(x, n)`, `ConfinedToColumnInBlock(y, n)`, `DifferentColumns(x, y)` → `DifferentBlocks(x, y)`

[22] `ConfinedToColumnInBlock(y, n)`, `DifferentColumns(x, y)`, `SameBlock(x, y)` → `Excluded(x, n)`

[23] `ConfinedToRowInBlock(x, n)`, `ConfinedToRowInBlock(y, n)`, `DifferentRows(x, y)` → `DifferentBlocks(x, y)`

[24] `ConfinedToRowInBlock(y, n)`, `DifferentRows(x, y)`, `SameBlock(x, y)` → `Excluded(x, n)`

[25] `ConfinedToColumnInBlock(x, n)`, `ConfinedToRowInBlock(y, n)`, `Excluded(y, n)`, `SameColumn(x, y)` → `DifferentBlocks(x, y)`

[26] `DifferentRows(x, y)`, `Forced(y, n)`, `SameColumn(x, y)` → `Excluded(x, n)`

[27] `ConfinedToColumnInBlock(y, n)`, `DifferentBlocks(x, y)`, `SameColumn(x, y)` → `Excluded(x, n)`

[28] `ConfinedToColumnInBlock(y, n)`, `ConfinedToRowInBlock(x, n)`, `Excluded(y, n)`, `SameRow(x, y)` $\rightarrow$ `DifferentBlocks(x, y)`

[29] `DifferentColumns(x, y)`, `Forced(y, n)`, `SameRow(x, y)` $\rightarrow$ `Excluded(x, n)`

[30] `ConfinedToRowInBlock(y, n)`, `DifferentBlocks(x, y)`, `SameRow(x, y)` $\rightarrow$ `Excluded(x, n)`